Sign in

pytorch / **pytorch** Public

Tensors and Dynamic neural networks in Python with strong GPU acceleration

pytorch.org

View license

88.1k stars **23.6k** forks Branches Tags Activity

Star Notifications

⚠ This commit does not belong to any branch on this repository, and may belong to a fork outside of the repository.

f3f23f9

Go to file Code

bartvm autopep8 E303 f3f23f9 · 9 years ago

cmake/FindCUDA	Add 'cmake/FindCUDA/' from commit '…	9 years ago
test	Add tests for copy and pickle + make C…	9 years ago
tools	Fixes for changes in libs	9 years ago
torch	autopep8 E303	9 years ago
.gitignore	Fixes for changes in libs	9 years ago
.travis.yml	adding requirements.txt	9 years ago
Makefile	Fixes for changes in libs	9 years ago
README.md	readme	9 years ago
breaking_changes.md	Add more Tensor and Storage methods	9 years ago
requirements.txt	adding requirements.txt	9 years ago

View all files

pytorch [alpha-0]

The project is still under active development and is likely to drastically change in short periods of time.
We will be announcing API changes and important developments via a newsletter, github issues and post a link to the issues on slack.
Please remember that at this stage, this is an invite-only closed alpha, and please don't distribute code further.
This is done so that we can control development tightly and rapidly during the initial phases with feedback from you.

Installation

```
pip3 install .
python3 setup.py build
python3 setup.py install
```

Communication

- github issues: bug reports, feature requests, install issues, RFCs, thoughts, etc.
- slack: general chat, online discussions, collaboration etc. https://pytorch.slack.com/ . If you need a slack invite, ping me at soumith@pytorch.org
- newsletter: no-noise, one-way email newsletter with important announcements about pytorch. You can sign-up here: http://eepurl.com/cbG0rv

Timeline

We will run the alpha releases weekly for 6 weeks. After that, we will reevaluate progress, and if we are ready, we will hit beta-0. If not, we will do another two weeks of alpha.

- alpha-0: Working versions of torch, cutorch, nn, cunn, optim fully unit tested with seamless numpy conversions
- alpha-1: Serialization to/from disk with sharing intact. initial release of the new neuralnets package based on a Chainer-like design
- alpha-2: sharing tensors across processes for hogwild training or data-loading processes. a rewritten optim package for this new nn.
- alpha-3: binary installs (prob will take @alexbw 's help here), contbuilds, etc.
- alpha-4: a ton of examples across vision, nlp, speech, RL -- this phase might make us rethink parts of the APIs, and hence want to do this in alpha than beta

- alpha-5: Putting a simple and efficient story around multi-machine training. Probably simplistic like torch-distlearn. Building the website, release scripts, more documentation, etc.
- alpha-6: [no plan yet]

The beta phases will be leaning more towards working with all of you, convering your use-cases, active development on non-core aspects.

pytorch vs torch: important changes

We've decided that it's time to rewrite/update parts of the old torch API, even if it means losing some of backward compatibility (we can hack up a model converter that converts correctly).
This section lists the biggest changes, and suggests how to shift from torch to pytorch.

For now there's no pytorch documentation.
Since all currently implemented modules are very similar to the old ones, it's best to use torch7 docs for now (having in mind several differences described below).

Library structure

All core modules are merged into a single repository.
Most of them will be rewritten and will be completely new (more on this below), but we're providing a Python version of old packages under torch.legacy namespace.

- torch (torch)
- cutorch (torch.cuda)
- nn (torch.legacy.nn)
- cunn (torch.legacy.cunn)
- optim (torch.legacy.optim)
- nngraph (torch.legacy.nngraph - not implemented yet)

0-based indexing

pytorch uses 0-based indexing everywhere.
This includes arguments to `index*` functions and nn criterion weights.

Under the hood, on the C side, we've changed logic on TH / THC / THNN / THCUNN to introduce a TH_INDEX_BASE compile-time definition to switch between 0 and 1 indexing logic.

New Tensor API

All methods operating on tensors are now out-of-place by default.

This means that although `a.add(b)` used to have a side-effect of mutating the elements in a, it will now return a new Tensor, holding the result.
All methods that mutate the Tensor/Storage are now marked with a trailing underscore (including `copy` -> `copy_`, `fill` -> `fill_`, `set` -> `set_`, etc.).
Most of math methods have their in-place counterparts, so an equivalent to `a.add(b)` in Lua is now `a.add_(b)` (or `torch.add(a, a, b)`, which is not recommended in this case)

CUDA module

All tensors have their CUDA counterparts in torch.cuda module.

There is no `torch.cuda.setDevice` anymore. By default always the 0th device is selected, but code can be placed in a `with` statement to change it:

```
with torch.cuda.device(1):
    a = torch.cuda.FloatTensor(10) # a is allocated on GPU1
```

Calling `.cuda()` on tensors no longer converts it to a GPU float tensor, but to a CUDA tensor of the same type located on a currently selected device.
So, for example: `a = torch.LongTensor(10).cuda() # a is a CudaLongTensor`

Calling `.cuda(3)` will send it to the third device.
`.cuda()` can be also used to transfer CUDA tensors between devices (calling it on a GPU tensor, with a different device selected will copy it into the current device).

```
a = torch.LongTensor(10)
b = a.cuda()  # b is a torch.cuda.LongTensor placed on GPU0
c = a.cuda(2) # c is a torch.cuda.LongTensor placed on GPU2
with torch.cuda.device(1):
    d = b.cuda() # d is a copy of b, but on GPU1
    e = d.cuda() # a no-op, d is already on current GPU, e is d == True
```

Also, setting device is now only important to specify where to allocate new Tensors. You can perform operations on CUDA Tensors irrespective of currently selected device (but all arguments have to be on the same device) - result will be also allocated there. See below for an example:

```
a = torch.randn(2, 2).cuda()
b = torch.randn(2, 2).cuda()
with torch.cuda.device(1):
    c = a + b                    # c is on GPU0
    d = torch.randn(2, 2).cuda() # d is on GPU1
```

In the near future, we also plan to use a CUDA allocator, which allows to alleviate problems with cudaMalloc/cudaFree being a sync point.
This will help us to not worry about using buffers for every intermediate computation in a

module if one wants to do multi-GPU training, for example.
See: torch/cutorch#443

Numpy integration

Because numpy is a core numerical package in Python, and is used by many other libraries like matplotlib, we've implemented a two-way bridge between pytorch and numpy.

```
a = torch.randn(2, 2)
b = a.numpy() # b is a numpy array of type corresponding to a
              # no memory copy is performed, they share the same storag
c = numpy.zeros(5, 5)
d = torch.DoubleTensor(c) # it's possible to construct Tensors from num
              # d shares memory with b - there's no copy
```

New neural network module

After looking at several framework designs, looking at the current design of `nn` and thinking through a few original design ideas, this is what we've converged to:

- Adopt a Chainer-like design
  - Makes it extremely natural to express Recurrent Nets and weight sharing
  - Each module can operate in-place, but marks used variables as dirty - errors will be raised if they're used again
- RNN example:

```
class Network(nn.Container):
    def __init__(self):
        super(Network, self).__init__(
            conv1=nn.SpatialConvolution(3, 16, 3, 3, 1, 1),
            relu1=nn.ReLU(True),
            lstm=nn.LSTM(),
        )

    def __call__(self, input):
        y = self.conv(input)
        y = self.relu1(y)
        y = self.lstm(y)
        return y

model = Network()
input = nn.Variable(torch.zeros(256, 3, 224, 224))

output = model(input)

loss = 0
for i in range(ITERS):
    input, target = ...
    # That's all you need for an RNN
    for t in range(TIMESTEPS):
```

```
        loss += loss_fn(model(input), target)
    loss.backward()
```

- Here, nn.Variable will have a complete tape-based automatic differentiation implemented
- To access states, have hooks for forward / backward (this also makes multi-GPU easier to implement)
  - This has the advantage of not having to worry about in-place / out-of-place operators for accessing .output or .gradInput
- When writing the module, make sure debuggability is straight forward. Dropping into pdb and inspecting things should be natural, especially when going over the backward graph.
- Pulling handles to a module after constructing a chain should be very natural (apart from having a handle at construction)
  - It's easy, since modules are assigned as Container properties
- Drop overly verbose names. Example:
  - SpatialConvolution → conv2d
  - VolumetricConvolution → conv3d

Multi-GPU

Proposed solutions need to address:

- Kernel launch latency
  - without affecting the user's code
- Implementation should be as transparent as possible
  - Should we expose DPT as:
    - Split
    - ParallelApply (scheduling kernels in breadth first order, to address launch latency)
    - Join
- In backward phase, send parameters as soon as the module finishes computation

Rough solution:

```
# This is an example of a network that has a data parallel part inside
#
#              B is data parallel
#     +->A+-->B+-+
#  +--+           +->D
#     +->C+------+
class Network(nn.Container):
    __init__(self):
        super(Network, self).__init__(
            A = ...,
            B = GPUReplicate(B, [0, 1, 2, 3]), # Copies the module onto
```

```
            C = ...,
            D = ...
        )

    __call__(self, x):
        a = self.A(x)
        c = self.C(x)
        a_split = Split(a) # a_split is a list of Tensors placed on dif
        b = ParallelApply(self.B, a_split) # self.B is a list-like obje
        d_input = Join(b + [c]) # gathers Tensors on a single GPU
        return self.D(d_input)
```

Each module is assigned to a single GPU.

For Kernel Launch Latency:

- Python threading
- Generators

For parameter reductions ASAP:

- In the forward pass, register a hooks on a every parameter which are evaluated as soon as the last backward is executed for that parameter. The hook will then “all-reduce” those parameters across GPUs
    - Problem with multiple forward calls - how do you know that the parameters won't be used anymore?
        - Well, last usage in backward graph = first usage in forward graph, so this should be straightforward

Multiprocessing

We plan to make it as straightforward as possible, to use pytorch in a multiprocessing environment.
For this, we plan to implement a .share() method for tensors that will enable them to be shared across processes seamlessly

Releases 58

PyTorch 2.6.0 Release Latest
on Jan 29

+ 57 releases

Packages 0

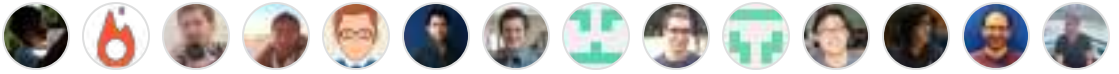
No packages published

Used by 667k



+ 667,202

Contributors 3,724



+ 3,710 contributors

Languages



Python 57.3% **C++** 34.6% **Cuda** 3.0% **C** 1.5% **Objective-C++** 1.1% **CMake** 0.7% **Other** 1.8%

Terms Privacy Security Status Docs Contact Manage cookies
Do not share my personal information

© 2025 GitHub, Inc.